

Composite Design Pattern

Study notes - structural pattern, first principles walkthrough with FAANG interview angle

1. The Problem (First Principles)

Real-life analogy: on a company org chart, an individual employee has a salary. A manager also has a salary - but also a team of people reporting to them, some of whom are themselves managers with their own teams. If HR asks "what's the total salary cost of this department?", you want to ask that question the same way whether you are pointing at a single intern or the VP at the top of a 200-person org.

In software this shows up as a **part-whole hierarchy** - a tree where some nodes are simple, indivisible things ("leaves") and other nodes are containers holding more of the same kind of thing ("branches"). A file system is the classic case: a File has a size; a Directory also has a size, but a Directory can contain more Files and more Directories, nested arbitrarily deep.

Without a formal pattern, client code ends up riddled with type-checks and duplicated recursive logic, because leaves and branches don't share a common interface the client can call blindly - every place that touches a tree item needs an if (isFile) ... else if (isFolder) ... check, recursively, at every level, forever.

2. Standard (GoF) Definition

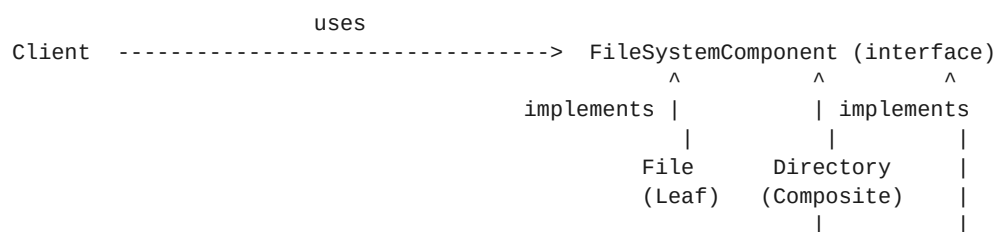
"Compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly."

It is a **structural** pattern. The key mechanical insight: give the leaf and the branch (container) the same interface. The container's implementation of that interface simply loops over its children and calls the same interface method on each - so the container never needs to know whether a child is itself a leaf or another container. A Composite's children are typed as Component, and a Composite IS ITSELF a Component - that's what makes it recursive.

3. Structure & Participants

Four roles, and one relationship that makes this pattern special - the recursive loop back onto itself:

- **Client** - your code. Only ever talks to Component, never checks whether it's holding a File or a Directory.
- **Component** (FileSystemComponent) - the shared interface both leaves and branches implement (e.g. getSize()).
- **Leaf** (File) - implements Component directly, no children. The base case of the recursion.
- **Composite** (Directory) - also implements Component, but additionally holds a collection of Component children. This is the recursive heart of the pattern: a child could be a File, or another Directory, nested arbitrarily deep, and the Directory does not need to know or care which.



```
+--- children: 0..* ---+
(Directory holds a list typed
as FileSystemComponent - can
be Files or more Directories)
```

Text rendering of the UML structure: Client -> Component (association), File -> Component and Directory -> Component (realization/implements), Directory -> Component 0.. (aggregation - the recursive "children" relationship at the heart of Composite).*

4. Code Example

The problem - without Composite

```
class File {
    private String name;
    private int size;
    public int getSize() { return size; }
}

class Directory {
    private List<File> files = new ArrayList<>();
    private List<Directory> subdirectories = new ArrayList<>();    // separate list, separate type!

    public int getTotalSize() {
        int total = 0;
        for (File f : files) total += f.getSize();
        for (Directory d : subdirectories) total += d.getTotalSize();    // manual recursion, duplicated
        return total;
    }
}
```

Every method that needs to walk the tree (getTotalSize, printAll, search, ...) repeats this same two-loop pattern: one loop for files, one loop for subdirectories, with an explicit recursive call for the subdirectory branch. Add a third kind of item and you need a third loop, in every method, everywhere.

The solution - with Composite

```
interface FileSystemComponent {
    int getSize();
    void print(String indent);
}

class File implements FileSystemComponent {    // Leaf
    private String name;
    private int size;
    public File(String name, int size) { this.name = name; this.size = size; }

    public int getSize() { return size; }
    public void print(String indent) {
        System.out.println(indent + "- " + name + " (" + size + " KB)");
    }
}

class Directory implements FileSystemComponent {    // Composite
    private String name;
    private List<FileSystemComponent> children = new ArrayList<>();    // ONE list, uniform type

    public Directory(String name) { this.name = name; }
    public void add(FileSystemComponent component) { children.add(component); }
```

```

    public int getSize() {
        int total = 0;
        for (FileSystemComponent child : children) {
            total += child.getSize();    // works whether child is a File or another Directory
        }
        return total;
    }

    public void print(String indent) {
        System.out.println(indent + "+ " + name + "/");
        for (FileSystemComponent child : children) {
            child.print(indent + " ");    // uniform recursive call, no type-checking
        }
    }
}

// Usage
Directory root = new Directory("root");
Directory docs = new Directory("docs");
docs.add(new File("resume.pdf", 120));
docs.add(new File("cover_letter.docx", 45));

Directory photos = new Directory("photos");
photos.add(new File("vacation.jpg", 3200));

root.add(docs);
root.add(photos);
root.add(new File("readme.txt", 2));

System.out.println("Total size: " + root.getSize() + " KB");
root.print("");

```

getSize() is called the exact same way on root (a deeply nested tree) as it would be on a single File. There is no if (isFile) anywhere. Adding a SymbolicLink later just means writing one new class implementing FileSystemComponent - zero changes to any existing traversal code.

5. Types / Variants of Composite

Type 1 - Transparent Composite (GoF's original form)

Put the child-management methods (add(), remove(), getChild()) directly on the shared Component interface - so File (the leaf) has an add() method too, even though adding a child to a file makes no sense.

Analogy: every button exists on the faceplate of every remote-controllable device - even a lamp has a "channel up" button molded into it, so you never have to check what device you're holding before pressing a button, but pressing "channel up" on a lamp obviously does nothing.

```

interface FileSystemComponent {
    int getSize();
    void add(FileSystemComponent c);    // present even on File!
    void remove(FileSystemComponent c);
}

class File implements FileSystemComponent {
    public void add(FileSystemComponent c) {
        throw new UnsupportedOperationException("Cannot add to a file");
    }
}

```

Pro: the client can treat every Component with zero casting, ever. Con: violates the Interface Segregation Principle - File is forced to implement methods meaningless for it, failing at runtime instead of compile time.

Type 2 - Safe Composite

Keep add()/remove()/getChild() only on the Composite class, not on the shared Component interface. File never has these methods at all.

Analogy: an individual employee's business card doesn't have a "direct reports" section - only a manager's card does. To assign someone a new report you must first confirm they are actually a manager before you can do it.

```
interface FileSystemComponent {
    int getSize();           // only the truly common operations
}

class Directory implements FileSystemComponent {
    private List<FileSystemComponent> children = new ArrayList<>();
    public void add(FileSystemComponent c) { children.add(c); } // only exists here
}
```

Pro: type-safe - File never exposes irrelevant methods, the compiler prevents nonsensical calls. Con: client code that builds the tree needs an instanceof check or downcast whenever it wants to add children - uniformity is lost at construction time even though it's preserved at traversal time.

Variant	Where add()/remove() live	Type safety	Real-life analogy
Transparent Composite	On the shared Component interface	Runtime errors possible	Every device has every button
Safe Composite	Only on the Composite class	Compile-time safe	Only a manager's card lists reports

Most real-world codebases lean toward **Safe Composite**, accepting a small amount of casting during tree construction in exchange for a cleaner, more honest interface.

6. Real-World Use Cases

- **File systems:** File and Directory, exactly as above - the textbook example.
- **GUI component trees:** Java Swing/AWT's Container (holds both individual Components and other Containers), or any web framework's DOM-like widget tree.
- **HTML/XML DOM:** every Node can contain child Nodes - text nodes are leaves, element nodes are composites.
- **Organization charts:** an individual contributor (leaf) vs a manager with reports who may themselves manage people (composite).
- **Menu systems:** a MenuItem (leaf) vs a Menu (composite containing more items and sub-menus).
- **Compiler ASTs:** a literal or variable reference (leaf) vs a compound expression like a binary operation (composite containing sub-expressions) - evaluate() is called uniformly at every node.

7. FAANG Interview Angle - Composite vs Its Cousins

- **vs Decorator:** both use recursive wrapping, but Decorator wraps exactly one object in a linear chain to add responsibilities (coffee wrapped in milk, wrapped in sugar). Composite represents a tree with arbitrary branching, where a node can have many children of mixed types. Decorator is a degenerate Composite with exactly one child and no true branching.
- **vs Iterator:** complementary, not competing. Composite gives you a uniform tree; Iterator is commonly layered on top to traverse that tree (depth-first, breadth-first) without the client knowing the tree's internal shape.
- **vs Visitor:** also commonly paired. If new operations (getSize(), print(), search(), export()...) keep being added directly to the Component interface, every leaf and composite must implement all of them, and the interface keeps growing. Visitor pulls those operations out into separate visitor classes using double dispatch - useful when operations multiply faster than node types do.
- **Common curveball:** "Isn't this just a recursive data structure - what does the pattern actually add?" Good answer: recursion alone doesn't require a shared interface. Composite's real contribution is formalizing that leaves and composites implement the exact same interface via polymorphism, so client code calls operations with no type-checking (Transparent) or only minimal, contained type-checking at construction time (Safe).
- **Another common question:** "How would you compute an aggregate like total size without violating single responsibility?" Good answer: keep the recursive aggregation logic inside the Composite's own method for simple cases; if the number of distinct operations over the tree keeps growing, that's the signal to refactor toward Visitor instead of continuing to bloat the Component interface.

8. Memory Map (Quick Recall)

COMPOSITE PATTERN

```
|
+-- PROBLEM: Leaves and branches of a tree need the same operation, but naive
|   code needs type-checks and duplicated recursive logic at every level
|   (analogy: asking total salary cost of an intern vs. a 200-person VP org
|   should be the same question, asked the same way)
|
+-- DEFINITION: Compose objects into tree structures to represent part-whole
|   hierarchies. Clients treat individual objects and compositions
|   of objects uniformly.
|
+-- 4 PARTICIPANTS
|   +-- Client    -> depends only on Component
|   +-- Component -> shared interface (getSize(), print()...)
|   +-- Leaf     -> File, implements Component, no children (base case)
|   +-- Composite -> Directory, implements Component AND holds 0..* Component
|                       children (recursive - the heart of the pattern)
|
+-- 2 VARIANTS
|   +-- Transparent Composite -> add()/remove() on shared interface,
|   |                           max uniformity, runtime errors possible
|   +-- Safe Composite       -> add()/remove() only on Composite class,
|   |                           type-safe, needs casting at tree-build time
|
+-- REAL EXAMPLES: file systems, GUI widget trees, DOM, org charts, menus,
|   compiler ASTs
|
+-- INTERVIEW DIFFERENTIATORS
|   +-- vs Decorator -> tree with branching, not a single linear wrap-chain
|   +-- vs Iterator  -> complementary: Iterator traverses the Composite tree
|   +-- vs Visitor   -> complementary: Visitor pulls growing operations out
|   |                   of the Component interface
|   +-- "just recursion?" -> no: the pattern's contribution is the shared
|   |                           interface via polymorphism, not recursion itself
```

One sentence to remember: Composite makes a single leaf and an entire subtree answer to the exact same call, so the client never has to ask which one it's holding.